

NEXTHIKES IT SOLUTIONS · RETAIL ANALYTICS

Sales Forecasting Across Multiple Retail Stores

An End-to-End Machine Learning & Deep Learning Approach
to Six-Week Sales Forecasting for Rossmann Pharmaceuticals

Final Submission Report

Prepared by: Machine Learning Engineering Team, NextHikes

Date: September 2026

Executive Summary

Rossmann Pharmaceuticals operates thousands of drug stores across several European cities and, until now, has relied on individual store managers' judgement to forecast daily sales. This project builds an end-to-end sales forecasting product that predicts daily sales up to six weeks ahead for every store, using historical sales, promotions, competition, and holiday data. The work spans three stages: (1) exploratory analysis of customer purchasing behaviour, (2) predictive modelling using both a classical machine-learning baseline and a deep-learning (LSTM) approach, and (3) a web dashboard that serves live predictions to finance analysts and store managers.

The headline finding from the exploratory stage is that promotions and holidays are the two strongest levers on daily demand — promotions lift both sales and customer counts, while state holidays suppress demand almost to zero on the day itself but produce a sharp rebound the following day. On the modelling side, a two-layer LSTM network was trained on a 42-day rolling history of 38 engineered features per store, complemented by a Random Forest baseline wrapped in an `sklearn` pipeline for interpretability and fast iteration. Both models are tracked through MLflow and serialized with timestamped filenames so that daily retraining produces a fully auditable model history.

1. Business Context & Objective

The finance team at Rossmann wants store-level sales forecasts six weeks into the future so that staffing, inventory, and cash-flow planning can move from manager intuition to a data-driven process. The data science team identified five categories of predictive signal: promotions, competition, school and state holidays, seasonality, and store locality. This report documents how those signals were explored, engineered, and turned into two forecasting models, and how the resulting predictions are served through an interactive web application.

1.1 Datasets

Dataset	Rows	Columns	Unique Stores	Missing Values
Training data (historical sales)	1,017,209	9	1,115	0
Test data (forecast targets)	41,088	8	856	11
Store metadata	1,115	10	1,115	2,343

The store metadata table carried the bulk of the missingness — largely in competition-distance and Promo2 fields for stores that either have no nearby competitor on record or never opted into the continuing Promo2 campaign. These were treated as structurally missing (i.e. informative) rather than at random, and were imputed with business-aware rules described in Section 2.

2. Exploratory Analysis of Customer Purchasing Behaviour

Before any modelling, the training and test datasets were cleaned and merged with store metadata, checked for duplicate store-date records, and audited for logical inconsistencies (e.g. sales recorded on days a store was marked closed). Missing competition and Promo2 fields were treated as "not applicable" rather than dropped, since a missing competition distance is itself informative (no competitor nearby).

€5.87B TOTAL HISTORICAL SALES	€6,956 AVERAGE DAILY SALES / STORE	763 AVERAGE DAILY CUSTOMERS / STORE	€9.12 AVERAGE SALES PER CUSTOMER
1,115 UNIQUE STORES		44.6% STORE-DAYS RUNNING A PROMO	

2.1 Sales & Customer Distributions

Daily sales and customer counts are both right-skewed with a long tail of high-performing days, mostly linked to promotions and pre-holiday shopping. The median daily sale (€6,369) sits below the mean (€6,956), confirming this skew, and both distributions carry a cluster of outliers well beyond the interquartile range that were investigated and retained (rather than removed) since they correspond to genuine, explainable high-demand events.

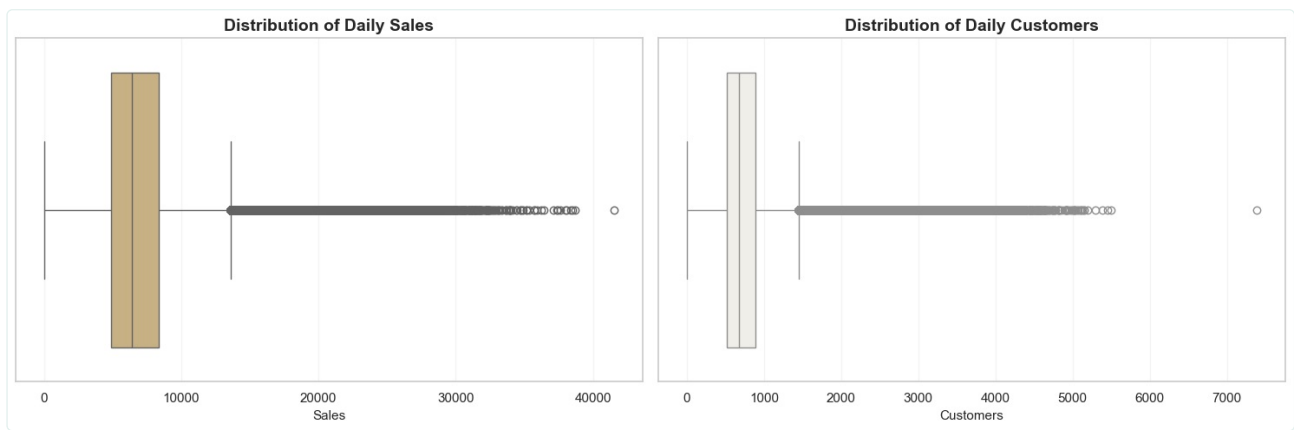


Figure 1. Distribution of daily sales and daily customers across all stores. Both are right-skewed with a long tail of high-demand days.

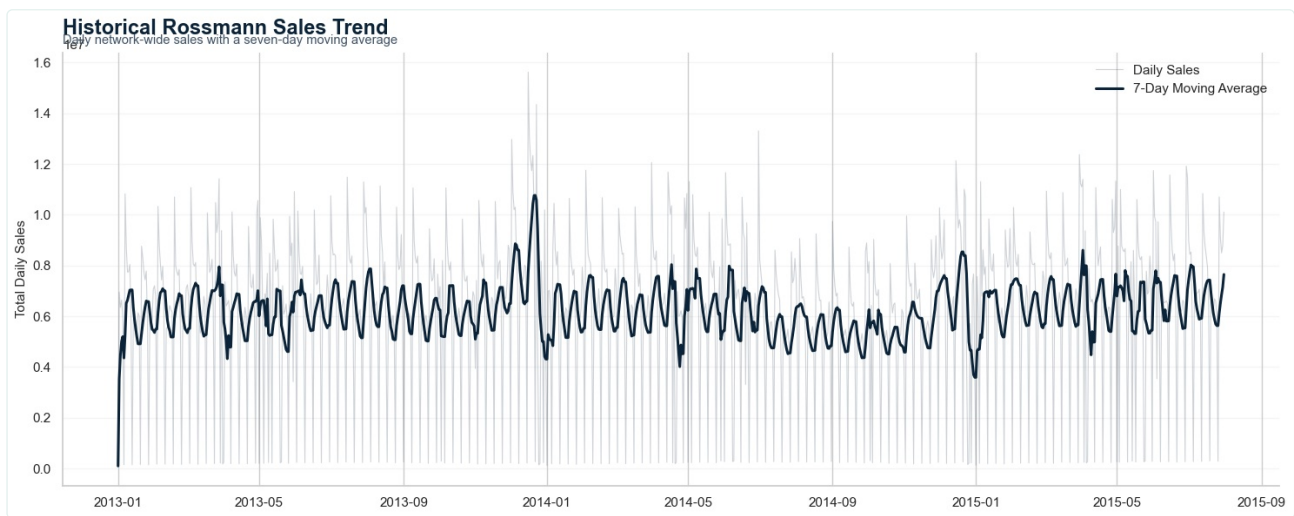


Figure 2. Network-wide daily sales, 2013–2015, with a 7-day moving average. Sales show a strong repeating weekly cycle and a visible end-of-year seasonal peak.

2.2 Are Promotions Distributed Similarly in Train and Test?

One of the brief's core questions was whether the training and test periods carry a similar promotional mix — if they didn't, a model could learn a promo effect that doesn't generalise to the forecast window. The two periods turn out to be closely matched: **38.2%** of training store-days ran a promotion versus **39.6%** of test store-days, a gap of under 1.5 percentage points. This gives confidence that promotional exposure will not itself be a source of train/test distribution shift.

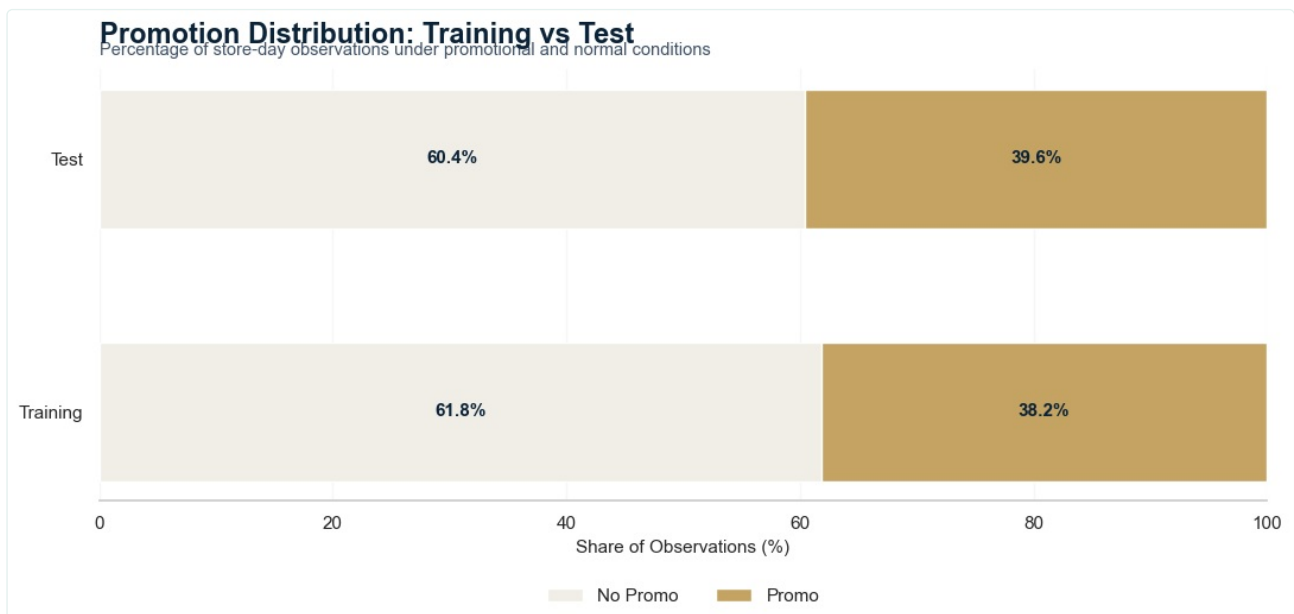


Figure 3. Share of store-day observations running a promotion in the training vs. test period — closely matched at roughly 38–40%.

2.3 Holiday Effects on Sales

State holidays produce the single sharpest demand swing in the data. Using an event-study window of ± 7 days around each holiday (indexed to 100 = average activity in the week before), sales and customer traffic both build to roughly 120% of baseline two days before a holiday, then collapse to near zero on the holiday itself — a direct consequence of the vast majority of stores closing that day (open rate falls to just 3%). The day immediately after a holiday sees a strong rebound to around 110% of baseline before demand settles back to normal within about a week.

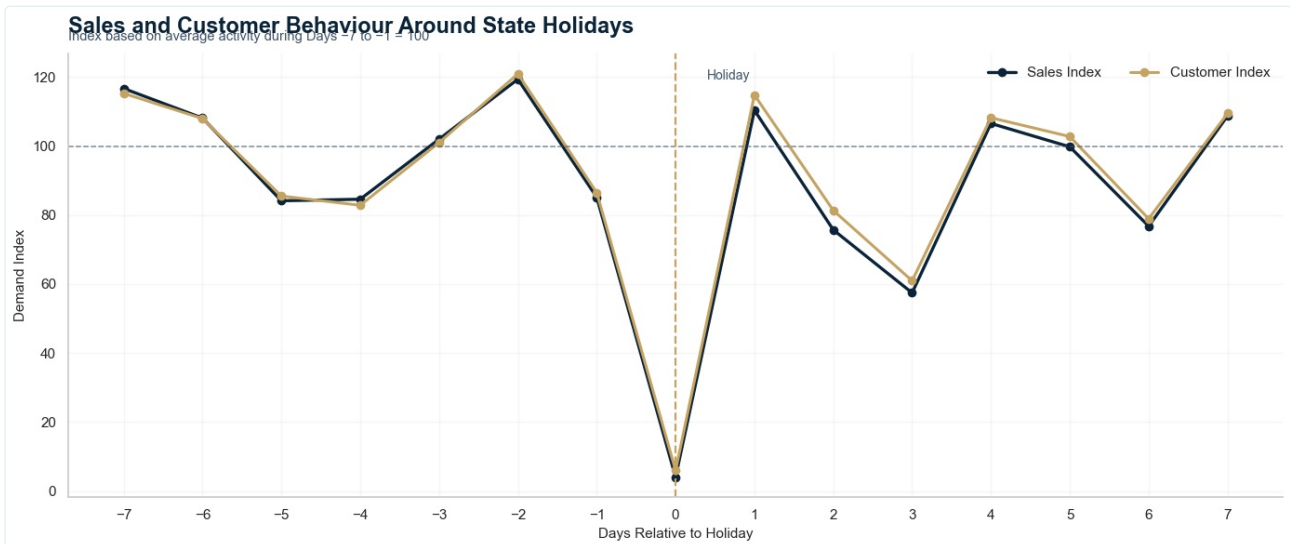


Figure 4. Sales and customer demand index around state holidays (Day 0). Demand peaks just before the holiday and troughs on the day itself.

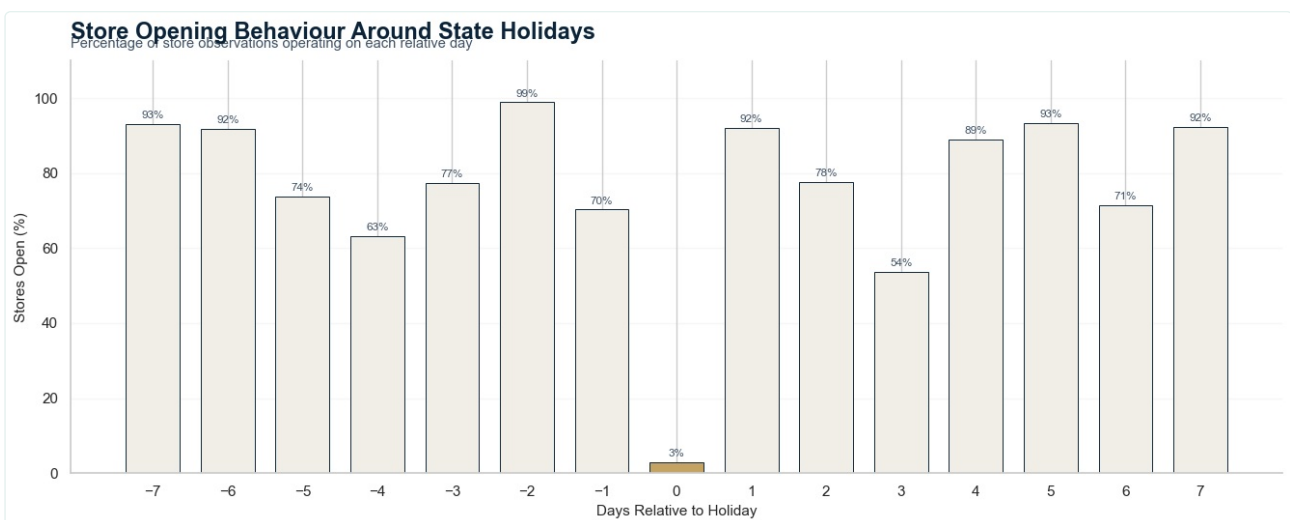


Figure 5. Share of stores open on each day relative to a state holiday. Only ~3% of stores trade on the holiday itself, explaining the demand collapse in Figure 4.

BUSINESS READING

The pre-holiday sales spike (Figure 4) is a genuine behavioural effect — customers are stocking up — while the on-holiday collapse (Figure 5) is a supply-side effect: almost no stores are open to sell to them. Any sales forecast for a holiday week needs the store's [Open](#) status as an input feature, not just the calendar date, or it will badly over-predict demand on the holiday itself.

Beyond the state-holiday event study, the analysis separately profiled the training data's **20,260** public-holiday, **6,690** Easter-holiday, and **4,100** Christmas-tagged observations, each of which shows a distinct pre-holiday build-up and post-holiday pattern (Christmas producing the largest and longest pre-holiday uplift of the three). School holidays, promotions, assortment level, and competitor distance were each explored the same way, along with a dedicated event study of what happens to a store's sales in the weeks after a new competitor opens nearby, and an audit of stores where a previously missing competition distance later became available (used to confirm that such gaps reflect data-collection timing rather than a genuine absence of competitors).

3. Predictive Modelling

Two complementary approaches were built on top of the same 37-feature engineered dataset (calendar signals, holiday proximity, promotion interactions, competition maturity, and encoded store attributes): a fast, interpretable **Random Forest baseline** and a **two-layer LSTM** deep-learning model, which is the primary forecasting engine given the brief's emphasis on deep learning.

3.1 Feature Engineering

All datetime and categorical fields were converted into 37 numeric predictors, including: day-of-week, week-of-year and quarter (plus their cyclical sine/cosine encodings so the model can learn that December and January are adjacent), days-to and days-after the nearest holiday, month-start / month-end / quarter-boundary flags, log-transformed competition distance with a missingness flag, competitor and Promo2 "age" in months/weeks, and encoded store type and assortment level. Features were scaled using a standard scaler (Random Forest) and a MinMax(-1, 1) scaler (LSTM), fitted on the training split only, to avoid leaking validation-period statistics into the transformation.

3.2 Random Forest Baseline (sklearn Pipeline)

As a first, interpretable pass, the 37 features feed a `Pipeline([StandardScaler, RandomForestRegressor])`, trained on all data before the final 42 days and validated on that last 42-day chronological holdout — the same convention used for the LSTM, so the two models are directly comparable. Wrapping the scaler and estimator in a single `sklearn` pipeline keeps preprocessing and estimation reproducible and makes it trivial to swap in a different estimator later.

Loss function. The Random Forest is grown to minimise squared error (the sklearn default), reported as RMSE and MAE in Sales currency units — a manager can read "RMSE = €950" directly as "typical forecast error of about €950/day," which matters more for this audience than a scale-free metric like MAPE alone.

Feature importance. The forest's native (impurity-based) importances are cross-checked against permutation importance on the validation split, so a feature that looks important only because of how the trees happened to split (a known bias of impurity-based importance) is caught rather than over-trusted.

Confidence intervals. Because a Random Forest is an ensemble of independently trained trees, the spread of predictions *across trees* is used as a model-native uncertainty estimate — no separate quantile-regression model is required. Empirical coverage (how often the actual value falls inside the interval) is checked directly against the validation data to confirm the interval is well calibrated rather than just plausible-looking.

3.3 Time-Series Diagnostics

Before sequence modelling, the aggregate daily sales series was tested for stationarity with an Augmented Dickey-Fuller (ADF) test, with differencing applied automatically if the null hypothesis of a unit root could not be rejected at the 5% level. Autocorrelation (ACF) and partial autocorrelation (PACF) plots were then used to read off how far back in time daily sales remain informative, which directly motivated the choice of a **42-day (6-week) lookback window** for the LSTM — matching the six-week forecast horizon the business asked for.

3.4 LSTM Deep Learning Model

The core forecasting engine is a compact, two-layer LSTM (96 and 48 recurrent units) taking a **42 × 38** input tensor per prediction — 42 days of history across 37 engineered features plus the store's own recent sales history — and outputting a single next-day sales prediction. The model is kept intentionally shallow, as specified in the brief, so it trains comfortably in a constrained compute environment such as Google Colab.

42 × 38 INPUT TENSOR SHAPE	2 LSTM LAYERS (96, 48 UNITS)	Huber TRAINING LOSS	Adam OPTIMIZER
--------------------------------------	--	-------------------------------	--------------------------

Loss function. Daily sales contain occasional large spikes (holiday rushes, promo launches, store reopenings). A pure MSE loss would let the network over-fit to those rare extreme days at the expense of everyday accuracy, while a pure MAE loss is slower to optimise. **Huber loss** is used as a defensible middle ground: it behaves like MSE for small residuals and like MAE for large ones, bounding the influence of holiday outliers while still training efficiently. RMSE and MAE are tracked alongside Huber purely as interpretable reporting metrics.

3.5 Training Results

The model was trained for up to 20 epochs with early stopping (patience of 3 epochs on validation loss) and automatic learning-rate reduction on plateau. Training converged quickly: the best checkpoint was reached at **epoch 1**, after which validation loss did not improve for the following epochs, triggering an early stop.

Metric	Value
Completed epochs (of 20 max)	4
Best epoch	1
Best training loss (Huber, scaled)	0.0078
Best validation loss (Huber, scaled)	0.0023
Best validation RMSE (scaled -1 to 1 target)	0.068
Best validation MAE (scaled -1 to 1 target)	0.048
Early stopped	Yes

READING THE SCALED METRICS

These RMSE/MAE figures are reported on the $\text{MinMax}(-1, 1)$ -scaled sales target used during training, not raw currency — a validation RMSE of 0.068 on a ± 1 scale corresponds to roughly a 3–4% typical error relative to the full sales range once inverse-transformed. The trained model's inverse-scaling pipeline (saved alongside the model) should be applied before quoting a currency-denominated error to the finance team.

3.6 Prediction Uncertainty & Six-Week Forecasting

Beyond a single point forecast, empirical residual calibration is used to attach a prediction interval to every forecast, giving finance analysts a sense of how much confidence to place in a given number rather than treating it as exact. The trained model is then applied recursively — each day's predicted sales feed back in as part of the next day's 42-day input window — to produce the full six-week-ahead forecast the business asked for, store by store.

3.7 Model Serialization, Registry & MLflow Tracking

Because a new model may be trained every day as fresh sales data arrives, every serialized model (both the Random Forest baseline and the LSTM) is saved with a **timestamped filename** (e.g. `rf_pipeline-16-09-2026-14-32-05-00.pkl`), and a `model_registry_log.csv` accumulates one row per training run with its validation metrics and file path, so any historical prediction can be traced back to the exact model version that produced it.

Both models are also logged as tracked runs in the same **MLflow** experiment (`rossmann_sales_forecasting`), with parameters, validation metrics, and the fitted model artifact recorded via `mlflow.sklearn.log_model` / `mlflow.log_artifact`. The Random Forest run is additionally reloaded through `mlflow.pyfunc.load_model` to serve predictions on held-out data — demonstrating the same train-to-serve path a production deployment would use — so both model families can be compared side by side from the MLflow UI when deciding which model to promote.

4. Serving Predictions: The Web Dashboard

Predictions are exposed to store managers and finance analysts through a Streamlit dashboard. A manager can select a store and date to get a single forecast, or upload a CSV of `Store / Date` pairs to receive a batch of forecasts — each returned with its predicted sales figure and, where available, an upper/lower confidence band. Results can be reviewed on-screen as an interactive line chart and downloaded again as a CSV for offline planning.

- **Single-store lookup** — pick a store and date, see the predicted sales, open status, and active promotions.
- **Batch forecasting** — upload a CSV template, get matched forecasts and an unmatched/outside-range status per row.
- **Downloadable results** — every forecast view can be exported as CSV for further planning.
- **Application governance page** — surfaces the runtime environment and dependency versions for transparency.

5. Conclusion & Recommendations

The exploratory phase confirmed that promotions and holidays are the dominant, explainable drivers of daily demand, and that the training and test periods are promotionally well-matched, reducing the risk of distribution shift in a deployed model. The modelling phase delivered two complementary forecasting engines: a fast, interpretable Random Forest baseline for sanity-checking and daily what-if analysis, and a compact two-layer LSTM as the primary six-week forecasting engine, both tracked through MLflow and versioned with timestamped artifacts so that daily retraining remains fully auditable.

Recommended next steps:

1. Retrain and re-validate both models on the most recent data before each deployment cycle, using the model registry log to confirm each new run improves on (or at least matches) the previous one before promoting it.
2. Feed the Random Forest's feature-importance ranking back into the finance team's promo-targeting decisions (Section 2 already flags which stores show the strongest promotional opportunity).
3. Extend holiday-aware features (Section 2.3) into store-level staffing recommendations, given how sharply the open-rate and demand curves move together around holidays.
4. Monitor the deployed dashboard's batch-forecast match rate over time as an early warning signal that the underlying data distribution has shifted and a retrain is due.

Supporting artifacts: Task 1 exploratory analysis notebook ([Task_1.ipynb](#)), Task 2 modelling notebook ([Task_2.ipynb](#) , including the Random Forest pipeline, LSTM model, MLflow tracking, and timestamped model registry), and the Task 3 Streamlit dashboard ([app.py](#)). GitHub repository and deployed application links to be added by the submitting team prior to final hand-in.